

Sécurité des applications et services web — cours complet

Niveau : Master 2 cybersécurité · **Volume** : 42 h · **Organisation** : 14 séances de 3 h, chacune composée de 90 min de théorie/démonstration et 90 min de pratique supervisée · **Code ECUE**, **calendrier et pondérations** : À confirmer.

Ce manuel accompagne un étudiant qui sait peu de choses du web jusqu'à une revue défendable d'une application locale. Il n'enseigne pas à collectionner des payloads : il apprend à relier une architecture, une hypothèse, un contrôle, une preuve et une décision. Toute reproduction offensive est limitée à [127.0.0.1](#), au réseau Docker isolé ShopLab ou à une plateforme pédagogique explicitement autorisée, avec comptes et données synthétiques.

1. La méthode qui relie les quatorze séances

Le fil conducteur est :

comprendre → **observer** → **reproduire de manière sûre** → **corriger** → **vérifier** → **documenter**

1. **Comprendre** : fixer scope, version, comportement attendu et menace. Sans attendu, une différence n'est pas encore un défaut.
2. **Observer** : capturer une baseline minimale : commande, statut, en-têtes pertinents, effet et identifiant de corrélation.
3. **Reproduire de manière sûre** : une variable à la fois, faible volume, données synthétiques, condition d'arrêt.
4. **Corriger** : agir sur la cause à la bonne frontière — requête paramétrée plutôt que blocklist SQL ; policy objet plutôt qu'UUID opaque.
5. **Vérifier** : rejouer exactement le test négatif, ajouter un test positif et observer les effets persistants/logs.
6. **Documenter** : scope, commit, temps UTC, commande, sortie expurgée, conclusion limitée et risque résiduel.

Une preuve distingue toujours : **observation** (« réponse 403 »), **interprétation** (« la policy semble refuser ») et **conclusion étayée** (« trois objets non possédés sont refusés, objet possédé accepté »). On ne conclut jamais « application sécurisée », mais « contrôles X et Y testés sur la version Z avec ces limites ».

Progression et compétences

Bloc	Séances	Compétences	Production intellectuelle
Autoriser et comprendre	S01–S03	C01–C03	contrat de test, trace de requête, lecture navigateur/serveur
Modéliser identité et accès	S04–S06	C04–C06	DFD, cycle de session, matrice sujet–action–objet
Maîtriser les entrées	S07–S09	C07, C10	source→sink, correction et retest des injections
Sécuriser API et infrastructure	S10–S12	C08–C10	contrat, proxy/TLS, privilèges et restauration DB
Industrialiser et répondre	S13–S14	C11–C12, C01	pipeline décisionnel, chronologie et rapport

2. S01–S03 — Du droit de tester à la réponse du navigateur

S01 : autorisation, scope et preuve (C01)

Une cible techniquement accessible n'est pas une cible autorisée. Le contrat de test nomme autorité, hôtes/ports, comptes, techniques, débit, créneau, exclusions, contacts et conditions d'arrêt. Pour ce cours : local/Docker autorisé ; Internet, données réelles, persistance et déni de service interdits. Une redirection hors scope est documentée sans être suivie.

La preuve minimale associe environnement et résultat :

```
date -u +%FT%TZ
git rev-parse --short HEAD
curl -i --max-time 5 http://127.0.0.1:8080/health
```

L'heure seule n'identifie pas le code ; le commit seul n'identifie pas l'exécution. Un hash établit qu'un fichier n'a pas changé depuis son calcul, pas qu'il était vrai. Cookies, tokens, mots de passe, données personnelles et clés privées sont expurgés. Le dossier contient le scope, l'environnement, les traces, les observations et un index SHA-256. Voir [S01](#), NIST SP 800-115 §§5.1–5.2 et WSTG 4.2.

S02 : le chemin d'une requête (C02)

Lire d'abord l'URL : schéma, hôte, port, chemin, query ; le fragment reste côté client. DNS résout un nom. TCP ou QUIC fournit le transport. TLS établit des clés et authentifie le pair selon la chaîne et le nom. HTTP exprime méthode, cible, champs et contenu. Le proxy sélectionne/limite/relaye. L'API parse, authentifie, autorise, valide, exécute puis utilise un rôle DB limité. La réponse revient par le chemin inverse.

```
GET /api/orders/42?view=full HTTP/1.1
Host: shop.test:8080
Accept: application/json
X-Request-ID: lab-s02-001
```

[Host](#) participe au routage ; [Accept](#) négocie une représentation ; l'identifiant suit l'opération sans devenir secret. Diagnostiquer par couche : erreur DNS ≠ refus de connexion ≠ erreur certificat ≠ 404 route ≠ 502 amont ≠ 500 application. [ping](#) ne valide ni TLS ni HTTP. La figure [chemin d'une requête](#) et [S02](#) détaillent RFC 3986, RFC 9110, RFC 8446 et RFC 9293.

S03 : HTTP et navigateur (C02, C03)

Les méthodes portent une sémantique : [GET](#) est safe ; [PUT](#) et [DELETE](#) sont idempotentes par intention. Les familles 2xx/3xx/4xx/5xx guident le client. [401](#) signifie authentification absente/invalid, [403](#) identité comprise mais refusée, [404](#) absence ou masquage, [422](#) contenu compris mais invalide, [429](#) limite, [502](#) amont.

Un cookie est un mécanisme navigateur ; une session est l'état de sécurité. Exemple :

```
Set-Cookie: __Host-session=<OPAQUE>; Path=/; Secure; HttpOnly; SameSite=Lax
```

[Secure](#) exige HTTPS ; [HttpOnly](#) interdit la lecture script ; [SameSite](#) réduit certains envois inter-sites ; aucun ne remplace l'autorisation. L'origine est le triplet schéma+hôte+port. La Same-Origin Policy limite surtout la lecture inter-origines. CORS est une permission déclarée par le serveur au navigateur ; il n'empêche pas [curl](#) ni un backend d'envoyer une requête. CSP limite les sources/contextes exécutables sans remplacer l'encodage. Quand [curl](#) réussit mais le navigateur bloque, examiner la politique navigateur ; quand les deux reçoivent 403, examiner le serveur. Voir [S03](#), RFC 9110, RFC 6454, Fetch et ASVS [v5.0.0-3.3.2](#), [3.3.4](#), [3.4.2-3.4.6](#).

3. S04–S06 — Menaces, identité et autorisation

S04 : transformer l'architecture en tests (C04)

Un DFD montre entités, processus, magasins, flux et frontières. Une frontière existe quand change propriétaire, privilège, identité ou mécanisme, pas seulement réseau. Pour chaque flux : producteur, consommateur, donnée, identité, confiance implicite et mode d'échec.

Un scénario est causal et testable : sujet A remplace [order_id](#) par l'objet de B ; l'API pourrait divulguer B faute de policy objet ; attendu $A \rightarrow A=200$, $A \rightarrow B=403/404$, anonyme=401. OWASP Top 10:2025 donne des catégories, ASVS 5.0.0 des exigences, WSTG 4.2 des méthodes. Catégorie → exigence → test → preuve, complété par le métier. STRIDE aide à poser des questions mais ne priorise pas. Vraisemblance, préconditions, impact et contrôles existants mènent à réduire, éviter, transférer ou accepter explicitement. Voir [S04](#).

S05 : mot de passe, MFA et session (C03, C05)

Le cycle d'identité comprend enrôlement, liaison, authentification, ajout/remplacement, récupération, révocation et fermeture. La récupération ne doit pas contourner le MFA. Deux connaissances ne font pas deux facteurs ; FIDO/WebAuthn résiste mieux au phishing qu'un code relayable.

Les mots de passe sont hachés par une fonction dédiée, salée et coûteuse, telle Argon2id ; jamais chiffrés pour relecture. Utiliser une blocklist de mots compromis, limiter les tentatives sans permettre un lockout malveillant et ne pas imposer de rotation périodique sans compromission.

Après login, une session de référence reçoit un identifiant CSPRNG d'au moins 128 bits, fait tourner l'ancien identifiant, applique timeout d'inactivité et durée absolue. Logout et incident invalident côté serveur. Un JWT signé n'est ni chiffré ni immédiatement révocable : valider algorithme autorisé, issuer, audience, dates, clé et état de révocation. La [figure du cycle](#), [S05](#), NIST SP 800-63B-4 et ASVS V6/V7 fournissent les critères.

S06 : la décision sujet–action–objet–contexte (C06)

Le serveur autorise chaque action sur chaque objet. RBAC structure des permissions par rôle ; ABAC emploie propriétaire, tenant, état, heure ou assurance. Cacher un bouton, connaître un UUID ou posséder un token valide ne prouve pas le droit sur l'objet.

```
order = db.get(Order, order_id)
if order is None:
    raise HTTPException(404)
if order.owner_id != principal.id and "orders:read:any" not in principal.permissions:
    raise HTTPException(403)
return public_order(order)
```

La version la plus robuste filtre souvent `id` et `owner_id` dès la requête. Tester collection, détail, modification, suppression, export et sous-ressources ; vérifier statut, corps, effet persistant et journal. Deny-by-default, moindre privilège, policy centralisée et cache invalidé forment la base. Voir [S06](#), ASVS V8, WSTG [WSTG-ATHZ-02/04](#) et OWASP API1:2023.

4. S07–S09 — Contrôler toutes les entrées selon leur destination

S07 : validation et SQLi (C07, C10)

Chemin, query, headers, cookies, JSON, fichier, message, base et API tierce sont des entrées. Canonicaliser une fois quand nécessaire, puis valider type, forme, taille, plage et logique. La destination décide la défense.

```
# Structure fixe, valeurs séparées
sql = text("SELECT id,total FROM orders WHERE owner_id=:uid AND status=:status")
rows = db.execute(sql, {"uid": principal.id, "status": status})
```

Le paramétrage protège les valeurs, pas un nom de colonne ; mapper les choix de tri à des constantes. Un ORM peut toujours exposer du SQL brut. Le rôle DB limité réduit l'impact sans remplacer le paramétrage. Avant/après : même entrée locale, nombre/ownership de lignes, correction source, test négatif et cas métier positif. Voir [S07](#), ASVS [v5.0.0-1.1.1](#), [2.2.1](#), [1.2.4](#) et WSTG [WSTG-INPV-05](#).

S08 : XSS, CSRF et CSP (C03, C07)

XSS transforme une donnée en HTML/JavaScript. Encoder au dernier moment selon le contexte ; préférer `textContent` à `innerHTML` ; accepter du HTML seulement avec sanitiseur maintenu et policy. CSP par nonce/hashees est une défense en profondeur :

```
Content-Security-Policy: default-src 'self'; script-src 'self' 'nonce-<ALEATOIRE>'; object-src 'none'; base-uri 'none'; frame-ancestors 'none'
```

CSRF exploite l'envoi automatique de l'autorité, notamment cookie. Une SOP qui bloque la lecture n'empêche pas toujours l'action. Les changements d'état utilisent une méthode adaptée, un token anti-CSRF lié à la session, SameSite selon usage, contrôle `Origin` complémentaire et re-authentification pour le critique. CORS n'est pas la correction CSRF. Vérifier sur deux origines locales : token absent/erroné refusé, valide accepté, sink rendu comme texte et CSP sans régression. Voir [S08](#), ASVS V1/V3, WSTG [INPV-01/02](#), [SESS-05](#), CSP3 et Fetch.

S09 : interpréteurs et ressources serveur (C07)

- **Commande** : préférer une bibliothèque ; sinon binaire fixe, `shell=False`, arguments séparés, allowlist, timeout, environnement et privilèges minimaux.
- **Traversal** : générer le nom physique ; résoudre le chemin, vérifier qu'il reste sous la racine ; maîtriser symlinks et courses.
- **Upload** : taille/quota, type détecté, nom serveur, stockage hors webroot, non-exécutable, quarantaine, retraitement et service en attachment.
- **SSRF** : allowlist schéma/hôte/port, résolution et plages refusées, validation de chaque redirect, timeout/taille, proxy de sortie ; contrôler les divergences de parseur et DNS rebinding.

Les retests utilisent marqueur, fichier leurre, upload inoffensif et service SSRF **dans le lab isolé**.

Bloquer seulement `127.0.0.1` ou l'apostrophe est une blocklist fragile. Voir [S09](#), ASVS

[v5.0.0-1.2.5](#), [1.3.6](#), [1.5.3](#), V5 et WSTG [INPV-12](#), [ATHZ-01](#), [BUSL-08/09](#).

5. S10–S12 — API, infrastructure et données

S10 : contrat et abus API (C06, C08)

Inventorier toutes les versions, routes, webhooks et opérations. OpenAPI décrit, mais seule la validation runtime impose. Utiliser des schémas dédiés par action avec `additionalProperties: false` ; ne jamais binder directement un modèle DB. Pour chaque requête : authentifier token (issuer, audience, dates, algorithme), autoriser fonction, objet et propriétés, puis valider logique.

Limiter corps, profondeur, pagination, durée, concurrence et fréquence. [429](#) doit être réellement atteint puis revenir à la normale. Ne faire confiance à l'IP transférée que depuis le proxy configuré. Une erreur stable expose code, message non sensible et request-id, jamais stack/SQL/token. Les logs emploient route normalisée et identité pseudonymisée. Voir [S10](#), OWASP API Top 10:2023, ASVS [v5.0.0-15.3.3/4](#), OpenAPI 3.1.1 et RFC 9457.

S11 : proxy, TLS et conteneurs (C09)

Le proxy doit écraser les forwarded headers entrants et l'application ne faire confiance qu'aux proxies configurés. TLS couvre chaque liaison selon menace ; vérifier chaîne, nom, période et renouvellement, jamais `-k` comme validation. HSTS demande gouvernance avant `includeSubDomains/preload`.

Les conteneurs utilisent image épinglée/minimale, utilisateur non root, filesystem read-only, capacités retirées, `no-new-privileges`, limites, réseaux séparés et ports host uniquement sur `127.0.0.1`. Pas de socket Docker ni mode privilégié. Vérifier faux header, certificat/nom, écriture interdite, health et statut applicatif inchangé. Voir [S11](#), ASVS V12/V13, RFC 7239, RFC 8446 et docs Nginx/Docker.

S12 : PostgreSQL limité et récupérable (C10)

Séparer owner, migrator, app et readonly. Retirer `PUBLIC`, accorder `CONNECT`, `USAGE` et opérations exactes ; penser aux séquences et aux default privileges du bon rôle créateur. Tester l'état effectif par connexion. RLS peut compléter l'application, mais certains rôles la contournent.

TLS DB avec validation `verify-full` répond au transit ; chiffrement disque répond au support ; l'accès autorisé voit les données. Une sauvegarde n'est prouvée qu'après restauration isolée, contrôles d'objets/données/droits et cleanup. Définir RPO, RTO, rétention, stockage séparé et protection de la copie. Les logs DB restent ciblés : « tout logger » peut copier les secrets. Voir [S12](#) et la documentation PostgreSQL current sur rôles, grants, RLS, `pg_dump`, `pg_restore`, SSL et logging.

6. S13–S14 — Décider automatiquement, répondre humainement

S13 : pipeline et exceptions (C11)

Un pipeline relie tests, secret scan, SAST, SCA/SBOM, build, image scan, environnement éphémère et DAST borné. Chaque outil a un angle mort ; zéro finding n'est pas zéro vulnérabilité. Un SBOM inventorie ; SCA rapproche composants et avis ; le triage confirme version, chemin, préconditions et impact.

Un secret Git exposé est révoqué/roté, pas seulement supprimé du dernier commit. Les jobs ont des permissions minimales ; les PR non fiables ne reçoivent pas les secrets. Un gate échoue sur finding non accepté. L'exception minimale est versionnée, prouvée, compensée, approuvée, possédée et expirante ; l'expiration bloque. Toujours lier rapport, commit, plateforme, outils et commande, en distinguant `testé ici`, `validé en CI` et `documenté – validation manuelle requise`. Voir [S13](#), NIST SSDF 1.1, ASVS V13/V15, SLSA, CycloneDX et SPDX.

S14 : observabilité, incident et rapport (C01, C12)

Logs = événements, métriques = agrégats temporels, traces = chemin distribué. Corréler sans stocker token, mot de passe, corps ou identifiant inutile. Temps UTC, format commun, accès, intégrité et rétention rendent l'observation exploitable. Une alerte lance une enquête : elle ne prouve pas une attaque.

Trier : confirmer, borner portée/impact, sévérité et propriétaire. Contenir limite maintenant ; éradiquer retire la cause ; restaurer revient à un état digne de confiance ; retour d'expérience améliore le système. Préserver avant action destructrice quand c'est sûr. Le rapport comporte résumé exécutif, scope/version, préconditions, reproduction minimale, observations expurgées, cause, impact, correction, retest, chronologie, index de preuves, confiance, limites et risque résiduel.

La [boucle incident](#) rejoint exactement la méthode du cours. Voir [S14](#), NIST SP 800-61 Rev. 3, ASVS V16, OWASP Logging et OpenTelemetry.

7. Cartes de décision réutilisables

Une entrée non fiable arrive : que faire ?

1. Identifier source, encodages et propriétaire.
2. Canonicaliser une fois si le format l'exige.
3. Valider structure/type/plage/limites métier.
4. Choisir le contrôle du sink : paramètres SQL, API sans shell, `textContent`/encodage contextuel, chemin généré, URL allowlistée.
5. Réduire privilèges, temps, taille et réseau.
6. Journaliser la décision sans donnée sensible.
7. Tester valeur légitime, valeur hostile et non-régression.

Un accès est demandé : que vérifier ?

1. Token/session valide et non révoqué.
2. Identité et assurance suffisantes pour l'action.
3. Fonction permise.
4. Objet/tenant/propriétaire permis.
5. Champs lisibles/modifiables permis.
6. Contexte et transition métier valides.
7. Décision et refus auditable.

Un finding arrive : que décider ?

1. Est-il dans le scope et lié à la bonne version ?
2. L'observation est-elle reproductible et non destructive ?
3. Quel actif, quelles préconditions et quel impact ?
4. Quelle cause et quelle frontière peuvent la supprimer ?
5. Quel test avant/après et quel test positif ?
6. Si différé : qui accepte, jusqu'à quand, avec quelle compensation ?
7. Le rapport formule-t-il limites et confiance ?

8. Référentiel de vérification final

À l'issue des 42 heures, une revue minimale de ShopLab doit montrer : scope local et stop conditions ; chemin DNS/TLS/HTTP/proxy/API/DB ; cookies/SOP/CORS/CSP/CSRF ; DFD et scénarios ; mot de passe/MFA/session ; 200/401/403 objet ; SQLi/XSS/commande/traversal/upload/SSRF corrigés et retestés ; OpenAPI/propriétés/tokens/429 ; proxy/TLS/conteneurs ; grants/backup/restore/audit ; pipeline avec gates/exceptions ; logs/incident/rapport ; reset et cleanup sans résidu.

Le [glossaire](#) fixe les termes. Les quatorze chapitres liés depuis le [chapitre-index](#) donnent les développements, exemples annotés, idées fausses, contrôles de connaissances et références exactes. Les sources de référence sont OWASP ASVS 5.0.0, WSTG 4.2, API Security Top 10:2023, Top 10:2025, RFC de l'IETF, NIST et documentations officielles des composants ; leur version doit être revue lors de toute future édition.

S01 — Éthique, autorisation, périmètre et preuve

Durée théorie/démonstration : 90 min · Compétence : C01

Prérequis et résultats attendus

Aucun outil offensif n'est requis. Il suffit de distinguer un fichier, une commande et une URL. À la fin, l'étudiant sait formuler une autorisation testable, vérifier qu'une cible est dans le périmètre, protéger une preuve, appliquer une condition d'arrêt et organiser un dossier reproductible.

1. Une capacité technique n'est pas une autorisation

Un test de sécurité modifie la charge, les données ou le comportement observable d'un système. L'absence d'intention malveillante ne donne aucun droit. Avant la première requête, quatre questions doivent recevoir une réponse écrite : **qui** autorise, **quoi** peut être testé, **quand**, et **comment** réagir en cas d'effet inattendu. Dans ce cours, l'autorisation permanente ne couvre que `127.0.0.1`, le réseau Docker isolé de ShopLab et une plateforme pédagogique explicitement désignée.

Un périmètre utile n'est pas « tester l'application ». Il nomme les actifs et exclusions : hôtes, ports, comptes synthétiques, techniques admises, créneau, débit maximal, personne de contact, données interdites. Une redirection vers un domaine public, une IP différente, un secret réel ou une donnée personnelle déclenche l'arrêt. On ne « vérifie pas juste une fois » hors périmètre.

```
Autorisé : http://127.0.0.1:8080/api/* ; comptes demo-* ; 10 req/s max
Interdit : toute autre adresse ; déni de service ; persistance ; données réelles
Arrêt    : sortie du scope, erreur 5xx répétée, ressource résiduelle, secret observé
Contact  : À confirmer
```

L'exemple transforme une permission vague en décisions mécaniques. Le débit est borné ; l'identité des données est précisée ; « interdit » couvre l'impact, pas seulement la destination.

2. Risque, vulnérabilité et preuve

Une vulnérabilité est une faiblesse. Le risque dépend aussi de la vraisemblance d'exploitation et de l'impact sur la confidentialité, l'intégrité, la disponibilité et parfois la traçabilité. Un message d'erreur n'est pas automatiquement une vulnérabilité ; une capture spectaculaire n'est pas automatiquement une preuve suffisante.

Une bonne preuve répond à cinq questions : quelle hypothèse, quelle version du système, quelle action exacte, quel résultat brut, quelle conclusion limitée ? Elle est minimale : on préfère l'identifiant synthétique d'une commande à l'export complet de la base. On expurge cookies, mots de passe et tokens tout en conservant le sens, par exemple `session=<REDACTED:sha256-prefix=ab12cd34>`.

```
date -u +%FT%TZ
git rev-parse --short HEAD
curl -i --max-time 5 http://127.0.0.1:8080/health
```

`date` fixe l'horloge de la preuve ; le commit identifie le code ; `--max-time` borne l'attente ; `-i` conserve statut et en-têtes. Une capture doit inclure la commande et non seulement `200 OK`.

3. Chaîne de conservation adaptée au laboratoire

La criminalistique formelle exige des procédures organisationnelles. Au laboratoire, on applique une version proportionnée : ne pas modifier l'original, travailler sur une copie, noter l'auteur et l'heure, calculer une empreinte, contrôler les accès. `sha256sum evidence.txt` sous Linux ou `shasum -a 256 evidence.txt` sous macOS détecte une modification ; l'empreinte ne prouve ni l'auteur ni la véracité initiale.

Une arborescence simple suffit :

```
preuves/S01/
├─ 00_scope.md
├─ 01_environnement.txt
├─ 02_requete_http.txt
├─ 03_observation.md
└─ index_sha256.txt
```

Le rapport sépare **observation** (« le serveur répond 403 »), **interprétation** (« la politique refuse probablement l'action ») et **conclusion vérifiée** (« le rôle `buyer` reçoit 403 pour trois objets non possédés »). Cette discipline évite de transformer une hypothèse en fait.

4. Boucle de travail sûre

1. **Comprendre** l'objectif et la frontière légale.
2. **Observer** une baseline sans charge agressive.
3. **Reproduire de manière sûre** sur données synthétiques, une variable à la fois.
4. **Corriger** la cause dans le code ou la configuration, pas seulement le symptôme.
5. **Vérifier** avec le même test négatif et un test fonctionnel positif.
6. **Documenter** versions, commandes, sorties expurgées et limites.

Le reset et le cleanup font partie du test. Un environnement qui laisse conteneur, volume ou donnée altérée n'est pas reproductible.

Idées fausses à corriger

- « Accessible sur Internet » ne signifie jamais « autorisé à tester ».
- `localhost` dans un conteneur désigne ce conteneur, pas forcément l'hôte.
- Une empreinte prouve l'intégrité depuis son calcul, pas l'origine du fichier.
- Un scan sans finding ne prouve pas l'absence de vulnérabilité.

Vérification des connaissances

1. Une redirection de ShopLab mène vers <https://example.org>. Que faire ? **Arrêter avant de suivre, conserver la redirection et signaler la sortie de scope.**
2. Pourquoi conserver un test positif après une correction ? **Pour vérifier que la fonction légitime n'a pas été cassée.**
3. Classifier « la réponse contient `Server: nginx` » : observation, interprétation ou risque ? **Observation ; le risque demande un contexte.**
4. Citer deux éléments à expurger. **Token de session, mot de passe, secret, donnée personnelle.**

Références normatives et officielles

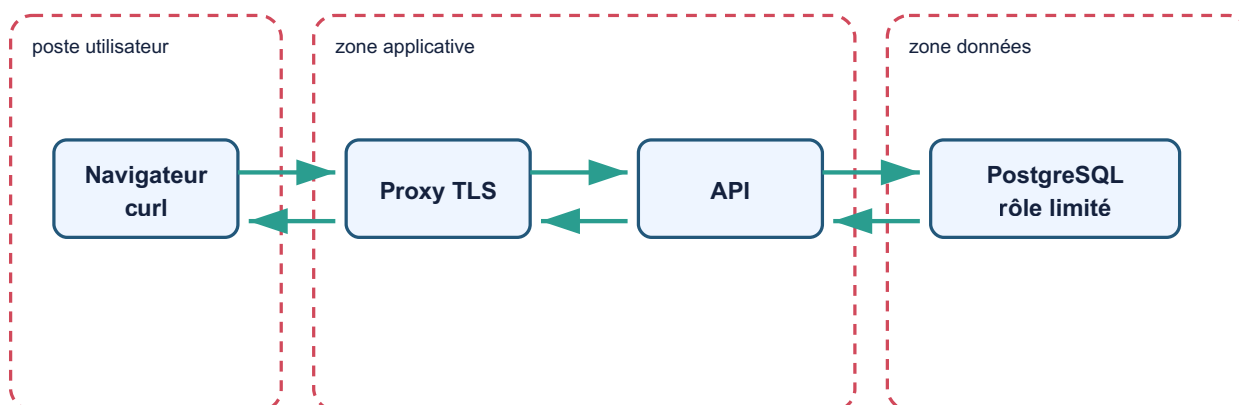
- OWASP, *Web Security Testing Guide 4.2*, scénario [WSTG-INFO-10](#) (cartographie d'architecture) et préface sur le périmètre : <https://wstg.owasp.org/v4.2/>
- NIST SP 800-115, sections 5.1–5.2, règles d'engagement et planification : <https://csrc.nist.gov/pubs/sp/800/115/final>
- NIST SP 800-61 Rev. 2, sections 3.2–3.4, cycle de gestion d'incident (retiré au profit de Rev. 3 mais utile pour la procédure détaillée) : <https://csrc.nist.gov/pubs/sp/800/61/r2/final>

S02 — DNS → transport → TLS → HTTP → proxy → API → base

Durée théorie/démonstration : 90 min · Compétence : C02

Prérequis et objectifs

Savoir lire une URL et lancer `curl`. L'étudiant saura raconter le trajet aller-retour d'une requête, associer une observation à la bonne couche et distinguer résolution de nom, connexion, protection TLS, routage HTTP, traitement API et requête SQL.



: une adresse ; TLS authentifie et protège le canal

Chaque frontière doit valider, autoriser, limiter et journaliser

1. Décomposer l'URL avant de déboguer

Dans `https://shop.test:8443/api/orders/42?view=full`, `https` est le schéma, `shop.test` l'hôte, `8443` le port explicite, `/api/orders/42` le chemin et `view=full` la requête. Le fragment `#details`, s'il existe, reste normalement dans le navigateur. Le schéma influence le port implicite et la sécurité attendue ; le nom doit devenir une adresse avant la connexion.

DNS répond à « quelle adresse pour ce nom ? ». TCP établit classiquement un canal ordonné ; HTTP/3 emploie QUIC sur UDP, donc « HTTP implique TCP » n'est plus universel. Une résolution réussie ne prouve pas qu'un port écoute. Une connexion réussie ne prouve pas que le certificat correspond au nom.

2. TLS authentifie le canal, pas l'utilisateur applicatif

Le client et le serveur négocient une version et des paramètres, le serveur présente une chaîne de certificats, le client vérifie notamment la confiance, la période et le nom, puis les deux dérivent des clés de session. TLS protège les octets en transit entre les extrémités TLS. Si un proxy termine TLS, la liaison proxy→API est une seconde liaison avec ses propres garanties.

```
openssl s_client -connect 127.0.0.1:8443 -servername shop.test </dev/null
```

`-connect` choisit l'adresse et le port ; `-servername` envoie le SNI et permet de sélectionner le bon certificat. Une sortie `Verify return code: 0` indique une chaîne validée avec le magasin utilisé, pas que l'application est sûre.

3. HTTP porte une intention auto-descriptive

```
GET /api/orders/42?view=full HTTP/1.1
Host: shop.test:8080
Accept: application/json
X-Request-ID: lab-s02-001
```

La ligne initiale porte méthode et cible. `Host` sélectionne l'hôte virtuel. `Accept` exprime une préférence de représentation. L'identifiant de corrélation permet de suivre la requête sans employer un token secret. Après la ligne vide viendrait le corps, absent ici.

Le proxy peut terminer TLS, imposer une taille, ajouter un identifiant, appliquer un rate limit puis acheminer `/api/` vers l'API. L'API parse l'entrée, authentifie, autorise, valide, exécute la logique et appelle la base avec un rôle limité. La réponse remonte en sens inverse ; un `502` au proxy n'est pas la même chose qu'un `500` généré par l'API.

4. Lire les erreurs par couche

Observation	Hypothèse prioritaire	Vérification bornée
<code>Could not resolve host</code>	DNS/hosts	<code>getent hosts</code> ou <code>dscacheutil -q host</code>
<code>Connection refused</code>	aucun listener/port erroné	<code>curl -v</code> , état du service
erreur de certificat	nom, temps, chaîne	<code>openssl s_client -showcerts</code>
<code>404</code>	routage ou ressource	comparer chemin/OpenAPI
<code>502 Bad Gateway</code>	proxy ne joint pas l'amont	logs proxy + health API
<code>500</code> avec transaction annulée	application/base	même request-id dans logs

Ne pas sauter directement à « le réseau est cassé ». Chaque test réduit l'espace des hypothèses.

5. Démonstration et boucle de sécurité

```
curl --verbose --resolve shop.test:8080:127.0.0.1 \  
-H 'X-Request-ID: lab-s02-001' http://shop.test:8080/health
```

`--resolve` remplace DNS pour ce nom/port sans modifier `/etc/hosts`; la destination reste locale. Observer d'abord la baseline, reproduire une erreur en changeant **un seul** élément (par exemple le chemin), restaurer, vérifier la réponse et documenter la couche responsable. Une correction « ouvrir tous les ports » masque le diagnostic et agrandit la surface.

Idées fausses fréquentes

- DNS ne transporte pas la requête HTTP ; il aide à trouver une adresse.
- TLS ne chiffre pas les données une fois déchiffrées au proxy ou dans les logs.
- `ping` ne teste ni TLS ni HTTP et peut être filtré alors que le service fonctionne.
- Un `200` affirme la réussite HTTP déclarée, pas la justesse métier ni l'autorisation.

Contrôle de compréhension

1. Le certificat est valide pour `shop.test`, mais la commande se connecte à `127.0.0.1` sans SNI/nom. Quelle couche échoue ? **Validation d'identité TLS.**
2. Où placer une corrélation utile ? **Dans un en-tête non secret propagé et dans les logs de chaque couche.**
3. Pourquoi la base ne devrait-elle pas être publiée sur l'hôte ? **Le client passe par l'API ; exposer la base contourne cette frontière.**

Références

- RFC 3986, sections 3–3.5, syntaxe URI : <https://www.rfc-editor.org/rfc/rfc3986>
- RFC 9110, sections 3, 4.2 et 7, ressources, schémas HTTP(S), routage : <https://www.rfc-editor.org/rfc/rfc9110>
- RFC 8446, sections 2 et 4, TLS 1.3 : <https://www.rfc-editor.org/rfc/rfc8446>
- RFC 9293, sections 3.4–3.10, TCP : <https://www.rfc-editor.org/rfc/rfc9293>
- curl, documentation officielle `--resolve` et mode verbose : <https://curl.se/docs/manpage.html>

S03 — HTTP, cookies, origine, CORS et politiques navigateur

Durée théorie/démonstration : 90 min · Compétences : C02, C03

Prérequis et résultats

Comprendre le trajet S02. À la fin, l'étudiant annote une transaction HTTP, explique statuts et cache, calcule une origine, décrit l'envoi d'un cookie et sépare SOP, CORS, CSP et contrôle serveur.

1. Méthode, statut et représentation

Une méthode exprime une sémantique. `GET` lit ; `POST` soumet un traitement ; `PUT` remplace classiquement une ressource ; `PATCH` modifie partiellement ; `DELETE` demande une suppression.

« Safe » signifie sans modification demandée par la sémantique, pas sans logs. « Idempotent » signifie que répéter la même intention a le même effet voulu, pas que les réponses sont identiques.

Les familles de statuts guident le client : 2xx réussite, 3xx redirection, 4xx problème relatif à la requête, 5xx incapacité du serveur. Choix utiles : `201` + `Location` après création, `204` sans corps, `400` syntaxe/validation générale, `401` authentification absente/invalides avec éventuel `WWW-Authenticate`, `403` refus, `404` ressource absente ou volontairement masquée, `409` conflit d'état, `422` contenu compris mais invalide, `429` limite dépassée, `500` erreur interne, `502/503/504` erreurs d'amont/disponibilité.

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Cache-Control: no-store
Set-Cookie: session=opaque; Path=/; Secure; HttpOnly; SameSite=Lax
Content-Security-Policy: default-src 'self'; object-src 'none'; base-uri 'none'

{"order_id":42,"status":"paid"}
```

`Content-Type` fixe l'interprétation ; `no-store` interdit le stockage de cette réponse ; le cookie est limité au transport HTTPS (`Secure`), inaccessible au JavaScript (`HttpOnly`) et soumis à une politique inter-site (`SameSite`). CSP contraint les contenus du document ; elle ne protège pas l'API contre une requête directe.

2. Cookie et session ne sont pas synonymes

Un cookie est un mécanisme de stockage/envoi du navigateur, sélectionné par domaine, chemin, sécurité, SameSite et expiration. Une session est l'état de sécurité associé. Le cookie peut ne contenir qu'un identifiant opaque ; l'état réside au serveur. Inversement, un token placé dans un en-tête n'est pas automatiquement plus sûr : un stockage JavaScript est exposé à XSS.

`Domain` absent crée un cookie host-only, souvent préférable. `Path` organise l'envoi mais n'est pas une barrière d'autorisation. Le préfixe `__Host-` impose `Secure`, `Path=/` et absence de `Domain` dans les navigateurs conformes.

3. L'origine et la Same-Origin Policy

Une origine est le triplet **schéma + hôte + port**. Les pages `https://shop.test/a` et `https://shop.test/b` partagent une origine ; `http://shop.test`, `https://api.shop.test` et `https://shop.test:8443` sont différents.

La SOP empêche principalement un script d'une origine de lire arbitrairement les ressources d'une autre. Elle n'empêche pas toutes les émissions : formulaires, images et certaines requêtes peuvent partir. C'est précisément pourquoi CSRF existe.

CORS permet au **serveur cible** d'autoriser le navigateur à exposer une réponse à une origine. Pour une requête non « simple », le navigateur envoie un preflight `OPTIONS`. `Access-Control-Allow-Origin: *` ne convient pas à une réponse sensible avec credentials. Réfléter aveuglément `Origin` revient à autoriser tout demandeur.

```
OPTIONS /api/orders HTTP/1.1
Origin: https://student.shop.test
Access-Control-Request-Method: POST
Access-Control-Request-Headers: Content-Type, X-CSRF-Token
```

Le serveur doit comparer l'origine à une allowlist exacte et répondre avec les méthodes/en-têtes nécessaires. CORS n'empêche ni `curl`, ni un service backend, ni un attaquant d'envoyer une requête : l'autorisation métier reste côté serveur.

4. Observer sans confondre les plans

Dans DevTools, l'onglet Network montre requête, réponse, initiateur et timing. L'onglet Application montre le stockage ; la console montre les blocages navigateur. Avec `curl`, on observe le serveur sans SOP. Si `curl` réussit et le navigateur bloque, chercher une politique navigateur. Si les deux obtiennent `403`, chercher l'authentification/autorisation serveur.

Boucle : comprendre la politique attendue ; capturer baseline ; reproduire sur deux origines locales ; corriger l'allowlist ou le header ; vérifier origine permise **et** origine refusée ; documenter le triplet complet et les en-têtes, sans cookie réel.

Confusions à éviter

- CORS n'est ni pare-feu ni mécanisme d'authentification.
- `HttpOnly` empêche la lecture JavaScript, pas l'envoi automatique ni CSRF.
- CSP atténue certaines conséquences XSS ; elle ne remplace pas l'encodage contextuel.
- Un statut n'est pas une preuve de contrôle : vérifier le corps, l'effet et les logs.

Questions de contrôle

1. `https://a.test` et `https://a.test:443` ont-ils la même origine ? **Oui, le port par défaut est 443.**
2. Pourquoi tester CORS avec un navigateur ? **Parce que le navigateur applique la décision d'exposition.**
3. Quel attribut réduit le vol de cookie par script ? `HttpOnly`, sans corriger la XSS elle-même.
4. `Path=/admin` remplace-t-il une autorisation ? **Non.**

Références

- RFC 9110, sections 9 et 15, méthodes et statuts : <https://www.rfc-editor.org/rfc/rfc9110>
- RFC 6265, sections 4–5, cookies HTTP ; mise à jour SameSite par RFC 6265bis en cours : <https://www.rfc-editor.org/rfc/rfc6265>
- RFC 6454, sections 4–7, concept d'origine : <https://www.rfc-editor.org/rfc/rfc6454>
- Fetch Standard, sections CORS protocol et CORS-preflight fetch : <https://fetch.spec.whatwg.org/>
- OWASP ASVS `v5.0.0-3.3.2`, `v5.0.0-3.3.4`, `v5.0.0-3.4.2` à `3.4.6` : <https://github.com/OWASP/ASVS/tree/v5.0.0>
- OWASP WSTG 4.2 `WSTG-SESS-02`, `WSTG-CLNT-07`, `WSTG-CONF-07` : <https://wstg.owasp.org/v4.2/>

S04 — Actifs, flux, frontières et modèle de menace

Durée théorie/démonstration : 90 min · Compétence : C04

Point de départ et objectifs

Il faut savoir suivre la requête S02–S03. L'étudiant saura dessiner un DFD minimal, inventorier actifs et surface d'attaque, écrire un scénario d'abus, relier menace, contrôle ASVS et test WSTG, puis prioriser sans confondre OWASP Top 10 et checklist exhaustive.

1. Partir du système réel

Un modèle de menace est une hypothèse structurée sur ce qui peut mal tourner. Il ne commence pas par un scanner. On définit d'abord la version et le périmètre, puis :

- **entités externes** : navigateur, opérateur, fournisseur de paiement simulé ;
- **processus** : proxy, API, worker ;
- **magasins** : PostgreSQL, volume d'uploads, journaux ;
- **flux** : HTTP, SQL, sauvegarde ;
- **frontières de confiance** : Internet→proxy, proxy→API, API→base, CI→registre.

Une frontière n'est pas seulement un rectangle réseau. Elle existe dès que changent identité, propriétaire, privilège ou mécanisme. Le pipeline qui signe une image franchit une frontière même sur le même serveur.

2. Des actifs aux scénarios testables

Les actifs ShopLab incluent commandes, adresses synthétiques, disponibilité, capacité à rembourser, secrets de signature et journal d'audit. Pour chaque flux, demander : qui produit, qui consomme, quelle identité, quelle donnée, quelle confiance implicite, quel échec ?

Un scénario d'abus utile suit une forme causale :

```
Étant donné un acheteur authentifié,  
quand il remplace order_id par celui d'un autre compte,  
alors l'API pourrait renvoyer la commande faute de contrôle d'objet,  
ce qui expose une donnée confidentielle.  
Contrôle attendu : ownership vérifié côté serveur.  
Test : compte A lit A=200 ; A lit B=403/404 ; anonyme=401.
```

La menace mène à une exigence et à une preuve. « Risque d'IDOR » seul ne dit ni condition ni impact.

3. Utiliser les catalogues avec discernement

OWASP Top 10:2025 communique des catégories de risques courants. ASVS 5.0.0 énonce des exigences vérifiables à niveaux. WSTG 4.2 propose des scénarios de test. Ils se complètent : catégorie → exigence → méthode → preuve. Aucun ne remplace la connaissance métier.

STRIDE offre des questions de rappel : usurpation, altération, répudiation, divulgation, déni de service, élévation de privilège. Il ne donne pas automatiquement la priorité. Pour prioriser, décrire vraisemblance, préconditions, impact et contrôles existants. Une échelle qualitative définie vaut mieux qu'un score pseudo-précis.

Niveau	Critère pédagogique
Critique	impact majeur plausible, exploitable avec accès faible ; arrêt/release gate
Élevé	impact important ou précondition réaliste ; corriger avant livraison
Moyen	impact limité ou contrôle compensatoire solide ; planifier et suivre
Faible	durcissement mesuré ; accepter explicitement ou corriger

4. Atelier de modélisation

Comprendre : fixer la version de ShopLab. Observer : dessiner le flux normal et lister les identités techniques. Reproduire sûrement : faire une revue sur papier, pas une attaque externe. Corriger : déplacer la décision d'autorisation dans l'API, réduire un grant ou ajouter une limite. Vérifier : transformer chaque contrôle en test positif/négatif. Documenter : numéroter [TM-S04-01](#), propriétaire, échéance et risque résiduel.

Une menace est « traitée » par réduction, évitement, transfert ou acceptation explicite. « WAF » sans règle, propriétaire ni test n'est pas un traitement vérifiable.

Erreurs de raisonnement fréquentes

- Une liste OWASP n'est ni un modèle du système ni un classement universel.
- Un composant interne n'est pas automatiquement fiable.
- Le chiffrement ne corrige pas une autorisation absente.
- Un DFD très détaillé mais sans frontières ni identités est décoratif.

Vérification des connaissances

1. Le stockage de logs contient-il un actif ? **Oui : preuve, disponibilité et confidentialité des événements.**

2. Donner une frontière non réseau. **Un job CI accédant au coffre de secrets.**
3. Pourquoi écrire le test dès le scénario ? **Pour rendre le contrôle falsifiable et suivre la couverture.**
4. Qui accepte un risque ? **Le propriétaire autorisé du risque, pas silencieusement le testeur.**

Références

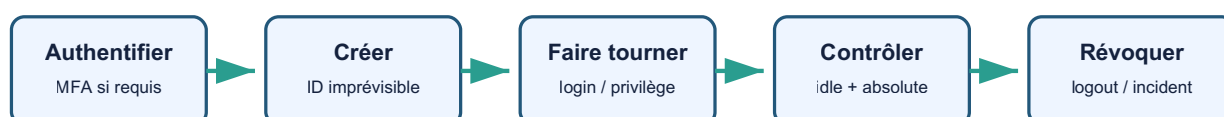
- OWASP Threat Modeling Cheat Sheet, processus, DFD et questions : https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html
- OWASP ASVS 5.0.0, notamment [v5.0.0-8.1.1](#) à [8.1.4](#) pour la documentation d'autorisation : <https://github.com/OWASP/ASVS/tree/v5.0.0>
- OWASP WSTG 4.2 [WSTG-INFO-10](#), cartographie de l'architecture : https://wstg.owasp.org/v4.2/4-Web_Application_Security_Testing/01-Information_Gathering/10-Map_Application_Architecture
- OWASP Top 10:2025, portée et catégories : <https://owasp.org/Top10/2025/>
- NIST SP 800-154, guide de threat modeling centré données : <https://csrc.nist.gov/pubs/sp/800/154/ipd>

S05 — Authentification, mots de passe, MFA et cycle de session

Durée théorie/démonstration : 90 min · Compétences : C03, C05

Prérequis et acquis visés

Connaître cookies, HTTPS et menaces. L'étudiant saura séparer identité, authenticateur et session ; concevoir stockage de mot de passe, MFA et récupération ; vérifier création, rotation, expiration et révocation ; comparer session opaque et JWT sans slogans.



À chaque requête : validité → identité → action → objet → contexte

1. Authentifier n'est qu'un début

Une identité revendiquée devient une identité authentifiée quand le vérificateur obtient une preuve suffisante du contrôle d'un authenticateur. La décision d'accès vient ensuite. Le cycle comporte inscription, liaison d'authenticateur, usage, ajout/remplacement, récupération, révocation et fermeture du compte. La récupération ne doit pas être plus faible que la connexion qu'elle restaure.

Les facteurs sont connaissance, possession et inhérence. Mot de passe + PIN reste une seule catégorie. Un TOTP résiste au rejeu grâce à sa courte fenêtre mais peut être hameçonné ; WebAuthn/ FIDO lie cryptographiquement l'opération à l'origine et apporte une meilleure résistance au phishing. Les codes de secours sont des secrets de récupération : usage unique, stockage haché et révocable.

2. Stocker et vérifier un mot de passe

Ne jamais chiffrer les mots de passe pour les relire. Employer une fonction dédiée, salée et coûteuse telle qu'Argon2id, avec paramètres mesurés et bibliothèque maintenue. Le sel unique n'est pas secret ; un *pepper* éventuel est un secret séparé, géré hors base.

```
from argon2 import PasswordHasher
from argon2.exceptions import VerifyMismatchError

ph = PasswordHasher()                # paramètres définis et benchmarkés au déploiement
stored = ph.hash(password)           # sel aléatoire inclus dans la chaîne encodée
try:
    ph.verify(stored, candidate)      # comparaison via la bibliothèque
except VerifyMismatchError:
    deny_with_generic_message()
```

La réponse d'échec ne révèle pas si le compte existe ; la limitation de tentatives tient compte du compte et du contexte sans permettre un verrouillage malveillant permanent. Les mots de passe soumis à création/changement sont comparés à une blocklist de mots compromis. Les rotations périodiques arbitraires poussent aux variantes prévisibles ; on change lors d'une compromission ou demande explicite.

3. Créer, faire tourner et terminer la session

Après authentification, le serveur crée un identifiant aléatoire d'au moins 128 bits pour une session de référence, stocke côté serveur identité, dates et état, puis pose un cookie `Secure; HttpOnly; SameSite=Lax/Strict` selon le cas. Il **fait tourner** l'identifiant après login et changement de privilège pour empêcher la fixation. Il impose expiration d'inactivité et durée absolue documentées.

Le logout invalide le serveur, pas seulement le cookie du navigateur. Le changement de mot de passe, la suppression d'un facteur et un incident peuvent révoquer toutes les sessions. Un JWT signé est vérifiable sans lecture d'état, mais plus difficile à révoquer immédiatement ; il nécessite audience, issuer, dates, algorithmes permis, rotation de clés et stratégie de révocation. JWT n'est pas chiffré par défaut.

```
POST /login HTTP/1.1
Content-Type: application/json

{"email":"alice@example.test","password":"<REDACTED>"}

HTTP/1.1 200 OK
Set-Cookie: __Host-session=<OPAQUE>; Path=/; Secure; HttpOnly; SameSite=Lax
Cache-Control: no-store
```

L'absence de `Domain` avec le préfixe `__Host-` limite l'injection depuis un sous-domaine. Le mot de passe et le cookie ne doivent jamais apparaître dans les captures partagées.

4. Vérifier le cycle complet

Comprendre la politique (MFA, timeouts, sessions parallèles). Observer login et cookie sans publier le secret. Reproduire localement : tentative générique, fixation, expiration simulée, logout. Corriger dans la gestion de session. Vérifier : ancien identifiant refusé après rotation/logout, nouveau accepté, action anonyme en 401, données toujours fonctionnelles. Documenter temps UTC, comptes synthétiques et identifiants hachés.

Matrice minimale :

Événement	Résultat attendu
échec login compte existant/inexistant	message et timing non discriminants
login réussi	nouvel identifiant, ancien invalide
élévation admin	rotation + re-authentification adaptée
timeout inactif/absolu	session refusée côté serveur
logout	rejeu de l'ancien cookie impossible
perte MFA	récupération vérifiée et facteur perdu révoqué

Idées fausses

- Base64 et chiffrement réversible ne sont pas du hachage de mot de passe.
- Deux secrets « connus » ne font pas du MFA.
- [Secure](#) n'empêche ni vol côté serveur ni CSRF.
- Une signature JWT prouve l'intégrité/émetteur attendu après validation complète, pas la confidentialité ni l'autorisation courante.

Contrôle des connaissances

1. Quand faire tourner une session ? **Après authentification et changement de niveau de privilège, entre autres.**
2. Pourquoi invalider côté serveur ? **Supprimer l'autorité, même si une copie du cookie subsiste.**
3. Quelle faiblesse d'un TOTP face à FIDO ? **Il peut être relayé/phishé.**
4. Pourquoi un message générique ? **Réduire l'énumération de comptes.**

Références

- NIST SP 800-63B-4 (final, juillet 2025), sections 3–4, authenticateurs, événements de cycle et session : <https://pages.nist.gov/800-63-4/sp800-63b.html>
- OWASP ASVS [v5.0.0–6.1.1](#), [6.2.1–6.2.10](#), [6.3.3](#), [6.4.3–6.4.4](#), [7.1.1](#), [7.2.1–7.2.3](#), [7.3.1](#), [7.4.1](#) : <https://github.com/OWASP/ASVS/tree/v5.0.0>

- OWASP Password Storage Cheat Sheet, Argon2id et paramètres : https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OWASP WSTG 4.2 [WSTG-ATHN-03](#), [WSTG-ATHN-07](#), [WSTG-SESS-01](#), [WSTG-SESS-03](#), [WSTG-SESS-06](#) : <https://wstg.owasp.org/v4.2/>
- RFC 7519, sections 4 et 7, JWT : <https://www.rfc-editor.org/rfc/rfc7519>

S06 — Autorisation, moindre privilège et BOLA

Durée théorie/démonstration : 90 min · Compétence : C06

Prérequis et résultats

Comprendre identité et session (S05). L'étudiant saura modéliser une décision sujet–action–objet–contexte, distinguer RBAC et ABAC, appliquer deny-by-default et moindre privilège, puis prouver une correction BOLA avec 200/401/403 ou 404 cohérents.

1. Une décision sur chaque action et chaque objet

L'interface peut cacher un bouton, mais seul le serveur décide. Une règle complète évalue : **sujet** (utilisateur/service), **action** (`read`, `refund`), **objet** (commande 42), **contexte** (tenant, état, heure, assurance). L'authentification seule répond uniquement au premier élément.

RBAC associe des permissions à des rôles : pratique pour `buyer`, `support`, `admin`. ABAC combine des attributs : propriétaire, tenant, état de commande. Dans ShopLab, « un acheteur lit ses commandes » est ABAC même si le rôle `buyer` permet l'accès général à la fonction.

```
def get_order(order_id: int, principal: User, db: Session):
    order = db.get(Order, order_id)
    if order is None:
        raise HTTPException(404)
    if order.owner_id != principal.id and "orders:read:any" not in principal.permissions:
        raise HTTPException(403)
    return public_order(order)
```

L'objet est chargé puis autorisé selon propriétaire ou permission explicite. Une approche encore plus robuste filtre dès la requête (`WHERE id=:id AND owner_id=:principal`) et traite l'absence sans fuite. `public_order` réduit les champs retournés ; autorisation objet et exposition de propriétés sont deux contrôles distincts.

2. BOLA et IDOR

Une référence directe (`/orders/42`) est normale. Elle devient un problème si le serveur croit que connaître 42 suffit. BOLA décrit l'échec de contrôle au niveau objet ; IDOR est une manifestation courante par identifiant modifiable. Remplacer des entiers par UUID réduit la devinabilité, pas l'obligation d'autoriser.

La vérification contrôlée utilise deux comptes synthétiques :

Requête	Attendu
anonyme → commande A	401
A → commande A	200, champs permis
A → commande B	403 ou 404, aucun champ de B
support sans permission → B	refus
rôle explicitement autorisé → B	200, événement audité

Tester liste, détail, modification, suppression, export et sous-ressources. Une route corrigée n'implique pas que `/orders/42/invoice` l'est.

3. Moindre privilège et séparation des fonctions

Le moindre privilège concerne utilisateur, compte de service, processus et rôle DB. Les permissions sont minimales en action, objets, champs et durée. Une tâche de remboursement peut exiger une re-authentification et une permission dédiée ; le support ne doit pas devenir administrateur global.

Le *deny-by-default* signifie qu'une action non explicitement permise est refusée. Centraliser l'évaluation dans une fonction/policy réduit les oublis, mais la règle métier doit rester lisible et testable. Les caches d'autorisation doivent être invalidés après changement de rôle.

4. Boucle de correction

Comprendre la matrice attendue. Observer une baseline propriétaire. Reproduire localement en changeant un seul `order_id`, sans énumération massive. Corriger au point commun côté serveur. Vérifier tous les chemins avec 200/401/refus et tests de non-régression. Documenter sujet, action, objet, résultat, request-id et absence de donnée sensible.

Une preuve 403 isolée ne suffit pas : le même endpoint pourrait toujours renvoyer un objet dans le corps ou modifier la base avant le refus. Vérifier corps, effet persistant et log d'autorisation.

Idées fausses

- Un identifiant imprévisible n'est pas un contrôle d'accès.
- Le frontend n'est jamais l'autorité de décision.
- « Admin » ne doit pas signifier accès permanent à tout sans audit.
- 404 peut réduire l'énumération, mais la politique doit rester cohérente et journalisée.

Vérification des connaissances

1. Pourquoi tester la collection et le détail ? **Le filtrage de liste et l'autorisation objet peuvent diverger.**
2. Où se trouve la règle « même tenant » ? **Dans la décision serveur, avec un attribut non fourni comme vérité par le client.**
3. Quel triplet distingue 401/403/200 ? **Sans identité / identité sans droit / identité avec droit.**
4. Un UUID corrige-t-il BOLA ? **Non.**

Références

- OWASP ASVS [v5.0.0-8.1.1](https://github.com/OWASP/ASVS/tree/v5.0.0) à [8.1.4](#) et chapitre V8 (Authorization) : <https://github.com/OWASP/ASVS/tree/v5.0.0>
- OWASP WSTG 4.2 [WSTG-ATHZ-02](#), contournement d'autorisation, et [WSTG-ATHZ-04](#), IDOR : https://wstg.owasp.org/v4.2/4-Web_Application_Security_Testing/05-Authorization_Testing/
- OWASP API Security Top 10:2023, [API1:2023 Broken Object Level Authorization](#) : <https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/>
- RFC 9110, sections 15.5.2–15.5.5, 401/403/404 : <https://www.rfc-editor.org/rfc/rfc9110>

S07 — Validation, typage et injection SQL

Durée théorie/démonstration : 90 min · Compétences : C07, C10

Prérequis et acquis

Savoir lire un endpoint et une requête SQL simple. L'étudiant saura localiser les frontières d'entrée, distinguer validation, paramétrage et encodage, expliquer comment données et instructions se mélangent, corriger une SQLi et démontrer non-régression.

1. Toute entrée a une provenance et une destination

Entrées : chemin, query, en-têtes, cookies, JSON, fichiers, événements, lignes de base et réponses d'API tierces. « Interne » n'est pas synonyme de fiable. La validation vérifie forme, type, longueur, plage et cohérence métier au plus tôt ; la protection finale dépend de la destination.

Canonicaliser une seule fois avant validation quand une forme encodée est attendue. Une double décodification permet à `%252e%252e` de devenir tardivement `../`. Une allowlist décrit ce qui est attendu ; une blocklist de caractères dangereux est fragile et ne remplace pas un mécanisme structurel.

2. Comment naît une injection SQL

```
# Vulnérable : l'entrée modifie la grammaire SQL
sql = f"SELECT id,total FROM orders WHERE owner_id={user_id} AND status='{status}'"
rows = db.execute(sql)
```

Si `status` contient une quote et des opérateurs, le serveur SQL ne peut plus distinguer la donnée de l'instruction, car le programme a déjà construit une seule chaîne. Échapper manuellement est dépendant du dialecte, de l'encodage et du contexte.

```
# Corrigé : structure et valeurs restent séparées jusqu'au pilote
sql = text("SELECT id,total FROM orders WHERE owner_id=:uid AND status=:status")
rows = db.execute(sql, {"uid": principal.id, "status": status}).mappings()
```

Les paramètres protègent les **valeurs**, pas les identifiants SQL comme nom de colonne ou direction `ORDER BY`. Pour ceux-ci, mapper une valeur métier vers une constante connue :

```
SORTS = {"date": Order.created_at, "total": Order.total}
sort_column = SORTS.get(requested_sort)
```

```
if sort_column is None:
    raise HTTPException(422, "unsupported sort")
query = select(Order).order_by(sort_column.asc())
```

3. Défense en profondeur côté base

Le compte applicatif ne doit pas posséder la base, créer des rôles ou lire toutes les tables. Même avec paramétrage, un défaut d'autorisation ou une requête légitime excessive existe. Limiter les grants, la durée de requête et les résultats réduit l'impact. Ne pas renvoyer les erreurs SQL détaillées au client ; conserver un identifiant de corrélation dans une trace protégée.

Les procédures stockées ne sont sûres que si elles évitent à leur tour SQL dynamique concaténé. Un ORM réduit certains risques mais permet souvent requêtes brutes : vérifier l'usage concret.

4. Preuve avant/après sans payload destructeur

Dans ShopLab local, définir une baseline (`status=paid` renvoie seulement les lignes payées du compte A). Reproduire avec la chaîne pédagogique fournie par le TP, sans écriture ni extraction massive. Capturer statut, nombre de lignes et request-id, jamais un dump. Corriger à la source. Rejouer exactement le test négatif : il doit être rejeté (422) ou traité comme valeur littérale sans élargir le résultat. Puis rejouer les statuts légitimes.

```
Avant : status=<test local> → 200, 6 lignes dont owner_id différent
Après : même entrée → 422, code=input.invalid
Régression : status=paid → 200, 1 ligne du principal
```

Cette boucle prouve simultanément sécurité et fonctionnalité. Un WAF qui bloque la chaîne connue sans corriger la concaténation n'est pas la remédiation racine.

Idées fausses

- Valider un entier dans l'UI ne protège pas l'API.
- Supprimer les apostrophes n'est pas une stratégie générale.
- ORM ne signifie pas « aucune injection ».
- Une réponse vide peut provenir d'une erreur masquée ; inspecter statut et logs corrélés.

Questions de contrôle

1. Pourquoi ne peut-on pas paramétrer directement un nom de colonne ? **Les paramètres représentent des valeurs, pas la grammaire SQL ; utiliser un mapping allowlist.**
2. Quelle validation pour `quantity` ? **Type entier, plage métier, cohérence avec stock et limites.**
3. Quel test après correction ? **Même entrée hostile + cas légitimes + contrôle de l'effet en base.**
4. Le rôle DB limité remplace-t-il le paramétrage ? **Non, il réduit seulement l'impact.**

Références

- OWASP ASVS [v5.0.0-1.1.1](#), [v5.0.0-2.2.1](#), [v5.0.0-1.2.4](#) : <https://github.com/OWASP/ASVS/tree/v5.0.0>
- OWASP WSTG 4.2 [WSTG-INPV-05](#), test SQL injection : https://wstg.owasp.org/v4.2/4-Web_Application_Security_Testing/07-Input_Validation_Testing/05-Testing_for_SQL_Injection
- OWASP SQL Injection Prevention Cheat Sheet : https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- PostgreSQL, documentation des privilèges et [GRANT](#) : <https://www.postgresql.org/docs/current/ddl-priv.html>

S08 — XSS, CSRF, SameSite et CSP

Durée théorie/démonstration : 90 min · Compétences : C03, C07

Prérequis et résultats

Maîtriser origine, cookie et session. L'étudiant saura distinguer XSS réfléchi, stocké et DOM, choisir un encodage selon le contexte, expliquer CSRF, combiner token/SameSite/origin-check et déployer CSP comme défense en profondeur.

1. XSS : une donnée devient du programme

Une XSS apparaît quand une donnée non fiable atteint un contexte interprété comme HTML ou JavaScript. Réfléchi : l'entrée de la requête revient immédiatement. Stockée : elle persiste puis touche d'autres utilisateurs. DOM : du JavaScript client lit une source et écrit dans un sink dangereux.

```
// Dangereux : innerHTML interprète une grammaire HTML
result.innerHTML = location.hash.slice(1);

// Sûr si le besoin est du texte
result.textContent = location.hash.slice(1);
```

L'encodage dépend du contexte : texte HTML, valeur d'attribut, URL, CSS et JavaScript ont des grammaires différentes. Utiliser le moteur de templates avec auto-escaping et éviter d'insérer une donnée dans un script. La sanitisation HTML n'est requise que si le produit doit accepter un sous-ensemble HTML ; elle emploie une bibliothèque maintenue et une politique explicite.

2. CSP limite l'exécution résiduelle

Une politique de départ :

```
Content-Security-Policy: default-src 'self'; script-src 'self' 'nonce-r4nd0m'; object-src 'none';
base-uri 'none'; frame-ancestors 'none'
```

Le nonce aléatoire est différent à chaque réponse et ajouté seulement aux scripts de confiance. Éviter `'unsafe-inline'` qui annule une grande partie du bénéfice. Commencer éventuellement en `Content-Security-Policy-Report-Only`, analyser les violations puis appliquer. CSP ne corrige pas le sink ; elle limite certaines exploitations et dérives de chargement.

3. CSRF : l'autorité ambiante est envoyée à l'insu de l'utilisateur

Le navigateur joint souvent les cookies automatiquement. Un site tiers peut déclencher une requête vers ShopLab ; la SOP peut empêcher la lecture de la réponse tout en laissant partir l'action. Une action d'état ne doit pas utiliser `GET`.

Défenses combinées : token anti-CSRF imprévisible lié à la session et vérifié côté serveur ; cookie `SameSite=Lax` ou `Strict` selon les usages ; contrôle exact de `Origin/Referer` en complément ; re-authentification pour action critique. Pour API sans cookie, exiger un en-tête d'autorisation non ajouté automatiquement réduit le scénario, mais XSS ou stockage inadéquat restent des risques.

```
POST /profile/email HTTP/1.1
Cookie: __Host-session=<OPAQUE>
Origin: https://shop.test
Content-Type: application/x-www-form-urlencoded
X-CSRF-Token: <TOKEN_LIE_A_LA_SESSION>

email=alice%40example.test
```

Le serveur vérifie méthode, session, origine attendue et token. Un token présent mais non comparé, réutilisable entre sessions ou placé dans une URL n'est pas une protection correcte.

4. Boucle locale de vérification

Comprendre source, sink et autorité. Observer rendu normal et en-têtes. Reproduire sur deux origines locales avec chaîne inoffensive fournie par le TP (par exemple un marqueur DOM, jamais vol de cookie). Corriger le sink et la validation CSRF. Vérifier : texte affiché sans interprétation ; token manquant/erroné refusé ; token valide accepté ; CSP sans erreur inattendue. Documenter DOM pertinent, requêtes expurgées et policy exacte.

Confusions à éviter

- `HttpOnly` réduit le vol direct de cookie, mais XSS peut toujours agir au nom de l'utilisateur.
- CORS ne corrige pas CSRF : une requête « simple » peut partir.
- `SameSite` est précieux mais une politique adaptée et un token restent nécessaires selon les scénarios.
- Encoder à l'entrée corrompt les données et ne connaît pas encore le contexte de sortie.

Contrôle de connaissances

1. `textContent` ou `innerHTML` pour afficher un nom ? `textContent`.
2. Où générer et vérifier le token CSRF ? **Avec une source aléatoire sûre, lié à la session, vérifié côté serveur.**

3. CSP avec `'unsafe-inline'` est-elle idéale ? **Non ; préférer nonce/hashe et supprimer les inline non maîtrisés.**
4. Pourquoi `GET /delete` est problématique ? **GET est safe par sémantique et peut être déclenché/préchargé.**

Références

- OWASP ASVS `v5.0.0-1.1.2, 1.2.1, 1.2.3, 3.3.2, 3.4.3, 3.5.1-3.5.2` : <https://github.com/OWASP/ASVS/tree/v5.0.0>
- OWASP WSTG 4.2 `WSTG-INPV-01, WSTG-INPV-02, WSTG-SESS-05, WSTG-CONF-12` : <https://wstg.owasp.org/v4.2/>
- CSP Level 3, directives et nonce : <https://www.w3.org/TR/CSP3/>
- Fetch Standard, CORS et credentials : <https://fetch.spec.whatwg.org/>
- OWASP CSRF Prevention Cheat Sheet : https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

S09 — Injection de commande, traversal, upload et SSRF

Durée théorie/démonstration : 90 min · Compétence : C07

Prérequis et objectifs

Comprendre la notion source→sink de S07–S08. L'étudiant saura repérer quatre familles de destinations serveur, supprimer l'interpréteur inutile, canonicaliser un chemin, isoler un upload et contraindre une requête sortante, puis construire des retests sûrs.

1. Injection de commande : ne pas invoquer de shell

```
# Dangereux : le shell interprète les métacaractères
subprocess.run(f"convert {name} /tmp/out.png", shell=True)

# Meilleur : arguments séparés, binaire fixe, timeout, privilèges réduits
subprocess.run(
    ["/usr/bin/convert", safe_input_path, safe_output_path],
    shell=False, timeout=5, check=True,
)
```

La première décision doit être : peut-on utiliser une bibliothèque plutôt qu'un processus ? Sinon, chemin du binaire fixe, arguments séparés, allowlist, timeouts, environnement minimal, répertoire isolé et utilisateur non privilégié. Les quotes ajoutées à une chaîne ne rendent pas tous les shells sûrs.

2. Traversal : résoudre puis vérifier l'appartenance

Concaténer `UPLOAD_ROOT / user_name` permet `../` et variantes encodées. Le serveur génère de préférence le nom physique et stocke le nom original comme métadonnée. Pour une lecture contrôlée :

```
root = Path("/srv/uploads").resolve()
candidate = (root / requested_name).resolve()
if root not in candidate.parents:
    raise HTTPException(400, "invalid path")
```

Cette vérification suit la canonicalisation, mais les liens symboliques et courses TOCTOU exigent des API de fichiers sûres et une configuration qui interdit les symlinks non maîtrisés. Une allowlist d'identifiants internes est préférable à un chemin fourni.

3. Upload : contenu hostile et stockage non fiable

Vérifier taille totale, quota, extension autorisée **et** type détecté ; renommer aléatoirement ; stocker hors webroot ; retirer droit d'exécution ; servir en téléchargement avec type fixe et `Content-Disposition: attachment`; analyser si nécessaire ; retraiter les images avec une bibliothèque maintenue. Le nom et le `Content-Type` client sont des déclarations, pas des preuves.

Même un fichier « valide » peut épuiser CPU/disque ou exploiter un parseur. Appliquer limites de dimensions, temps et décompression. La quarantaine sépare réception et publication.

4. SSRF : le serveur ne doit pas devenir proxy universel

Une fonctionnalité d'aperçu d'URL donne au serveur sa position réseau et parfois son identité cloud. Valider une URL contre une allowlist de schémas (`https`), hôtes et ports ; résoudre puis refuser loopback, link-local, réseaux privés et adresses interdites selon le besoin ; contrôler chaque redirection ; imposer timeout, taille et nombre de redirects ; utiliser un proxy de sortie/DNS maîtrisé. Se méfier des divergences de parseur et du DNS rebinding.

```
Entrée autorisée : https://images.example.test/catalog/42.png
Refus : file:///... ; http://127.0.0.1/... ; hôte non allowlisté ; redirect hors allowlist
```

Dans le laboratoire, la reproduction reste sur services Docker synthétiques. On ne sonde jamais les métadonnées d'un cloud ou l'intranet réel.

5. Vérifier quatre corrections

Pour chaque famille : baseline légitime ; entrée de test non destructive ; correction racine ; même test négatif ; usage légitime ; logs. Un test de commande peut demander la création d'un marqueur **dans un volume temporaire du lab**, puis vérifier son absence après correction. Le traversal cherche un fichier leurre synthétique. L'upload emploie une signature inoffensive. La SSRF pointe un service leurre local.

Idées fausses

- Échapper pour Bash ne protège pas PowerShell ni un autre interpréteur.
- `basename()` seul ne traite pas toutes les règles de stockage et collisions.
- Extension `.jpg` et `Content-Type: image/jpeg` sont contrôlés par le client.
- Bloquer la chaîne `127.0.0.1` ne couvre ni IPv6, ni DNS, ni redirections.

Questions

1. Meilleure défense contre command injection ? **Supprimer le shell et employer une API/ bibliothèque ; sinon arguments séparés et contraintes fortes.**

2. Pourquoi stocker hors webroot ? **Éviter exécution/rendu direct sous contrôle de l'attaquant.**
3. À quel moment vérifier une redirection SSRF ? **À chaque saut, avant la connexion suivante.**
4. Pourquoi générer le nom de fichier ? **Découpler identifiant externe et chemin physique.**

Références

- OWASP ASVS [v5.0.0-1.2.5, 1.3.6, 1.5.3, 5.2.4, 5.2.6, 5.3.1-5.3.2](https://github.com/OWASP/ASVS/tree/v5.0.0) : <https://github.com/OWASP/ASVS/tree/v5.0.0>
- OWASP WSTG 4.2 [WSTG-INPV-12](https://wstg.owasp.org/v4.2/#WSTG-INPV-12) (command injection), [WSTG-ATHZ-01](https://wstg.owasp.org/v4.2/#WSTG-ATHZ-01) (directory traversal), [WSTG-BUSL-08](https://wstg.owasp.org/v4.2/#WSTG-BUSL-08)/[WSTG-BUSL-09](https://wstg.owasp.org/v4.2/#WSTG-BUSL-09) (uploads) : <https://wstg.owasp.org/v4.2/>
- OWASP SSRF Prevention Cheat Sheet : https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html
- Python, documentation officielle [subprocess](https://docs.python.org/3/library/subprocess.html#security-considerations) et considérations de sécurité : <https://docs.python.org/3/library/subprocess.html#security-considerations>

S10 — Sécuriser une API REST/JSON

Durée théorie/démonstration : 90 min · Compétences : C06, C08

Prérequis et résultats

Savoir lire HTTP, AuthN/AuthZ et validation. L'étudiant saura inventorier une API, contraindre schéma et propriétés, sécuriser tokens, BOLA et mass assignment, limiter les ressources, traiter les erreurs et produire des logs exploitables.

1. Le contrat est aussi une frontière

OpenAPI inventorie méthodes, chemins, paramètres, schémas, réponses et mécanismes de sécurité. Il permet une revue et des tests, mais une documentation ne force rien sans validation runtime. Maintenir toutes les versions, routes d'administration, endpoints mobiles et webhooks ; une ancienne version oubliée reste une surface.

```
PatchOrder:
  type: object
  additionalProperties: false
  properties:
    delivery_note: {type: string, maxLength: 300}
  required: [delivery_note]
```

`additionalProperties: false` refuse les champs inattendus. Le schéma de mise à jour n'expose pas `owner_id`, `role`, `price` ou `is_paid`. Réutiliser directement le modèle DB dans l'API favorise le mass assignment.

2. Cinq contrôles indépendants

1. **Authentifier** le token : signature/état, issuer, audience, expiration, algorithme autorisé.
2. **Autoriser la fonction** : cette identité peut-elle appeler `PATCH /orders` ?
3. **Autoriser l'objet** : possède-t-elle cette commande ?
4. **Autoriser les propriétés** : peut-elle lire/modifier chaque champ ?
5. **Valider la logique** : transition d'état, quantité, total et séquence sont-ils permis ?

Un token OAuth porte une délégation et une portée, pas une preuve que l'objet appartient à l'utilisateur. Un JWT correctement signé avec mauvaise `aud` doit être refusé. Les API keys identifient souvent une application et nécessitent rotation, portée et quotas ; elles ne remplacent pas toujours l'utilisateur.

3. Limiter consommation et automatisation

Limiter taille de corps, profondeur JSON, pagination, durée, concurrence et fréquence selon identité et opération. Le `429 Too Many Requests` peut inclure `Retry-After`. Le rate limit ne doit pas faire confiance à un `X-Forwarded-For` fourni directement par le client ; seul le proxy de confiance réécrit la valeur.

La limite protège disponibilité et abuse cases, sans remplacer autorisation. Prévoir une réponse contrôlée et observer les erreurs de dépendances. Une requête idempotente peut être retentée avec backoff ; une création nécessite clé d'idempotence ou prudence pour éviter doublons.

4. Erreurs et journaux

Une erreur client stable contient statut, code métier, message non sensible et request-id. Elle n'expose ni stack trace, SQL, chemin local ni token.

```
{"code":"order.forbidden","message":"Action non autorisée","request_id":"lab-s10-0042"}
```

Les logs enregistrent route normalisée, identité pseudonymisée, décision, latence, résultat et corrélation ; pas le corps complet par défaut. Journaliser les refus et contournements aide à distinguer bug, abus et attaque.

5. Boucle de revue API

Comprendre le contrat et l'inventaire. Observer une requête nominale. Reproduire localement : objet d'autrui, champ inattendu, token expiré/mauvaise audience, rafale bornée. Corriger validation/policy/limites. Vérifier matrice positive et négative, dont `429` réellement mesuré et retour à la normale. Documenter version OpenAPI, principal synthétique, request-id et limites configurées.

Idées fausses

- REST n'apporte aucun contrôle de sécurité automatiquement.
- OpenAPI documenté mais non appliqué ne bloque rien.
- JWT signé ne signifie pas audience correcte, token actif ou droit sur l'objet.
- Masquer une route dans l'UI ne la retire pas de la surface API.

Contrôle de connaissances

1. Quel défaut permet `{"role":"admin"}` dans un profil ? **Mass assignment / autorisation de propriété absente.**
2. Pourquoi normaliser la route dans les métriques ? **Éviter une cardinalité par identifiant et agréger utilement.**

3. Différence quota et rate limit ? **Budget sur une période/usage versus cadence/concurrence ; ils peuvent se combiner.**
4. Quel statut pour limite temporaire ? **429, avec indication de retry si pertinente.**

Références

- OWASP API Security Top 10:2023, [API1](#), [API3](#), [API4](#), [API8](#), [API9](#) : <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
- OWASP ASVS [v5.0.0-15.3.3](#) (mass assignment), [15.3.4](#) (adresse proxy fiable), [8.1.1-8.1.2](#), [10.3.2](#) : <https://github.com/OWASP/ASVS/tree/v5.0.0>
- OpenAPI Specification 3.1.1, Schema Object et Security Requirement Object : <https://spec.openapis.org/oas/v3.1.1.html>
- RFC 9457, *Problem Details for HTTP APIs* : <https://www.rfc-editor.org/rfc/rfc9457>
- RFC 6585, section 4, statut 429 : <https://www.rfc-editor.org/rfc/rfc6585>

S11 — Reverse proxy, TLS, certificats et conteneurs

Durée théorie/démonstration : 90 min · Compétence : C09

Prérequis et résultats attendus

Il faut savoir suivre une requête (S02), lire des en-têtes (S03) et raisonner sur une API (S10). À la fin, l'étudiant sait dessiner les terminaisons TLS, expliquer le cycle d'un certificat, établir la confiance envers les en-têtes `Forwarded/X-Forwarded-*`, durcir un proxy et un conteneur, puis vérifier que le durcissement ne masque pas une erreur applicative.

1. Le proxy est une frontière, pas un tuyau transparent

Un reverse proxy accepte la connexion cliente, sélectionne un virtual host, peut terminer TLS, normalise la requête, applique tailles/timeouts/limites puis ouvre une nouvelle connexion vers l'amont. Il possède donc deux points de vue : celui du client et celui du service. Le schéma, l'adresse et l'identité observés par l'API peuvent être ceux du proxy.

L'en-tête standard `Forwarded` et les conventions `X-Forwarded-For`, `X-Forwarded-Proto`, `X-Forwarded-Host` transmettent ce contexte. Mais une valeur fournie par Internet n'est pas fiable. À la frontière, le proxy doit supprimer ou remplacer les valeurs entrantes ; l'application ne les accepte que d'adresses proxy configurées. Sinon un client peut falsifier son IP, générer une URL de reset en `http`, contourner une limite ou polluer l'audit.

```
location /api/ {
    proxy_pass http://api:8000/;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header X-Forwarded-For $remote_addr;
    proxy_set_header X-Request-ID $request_id;
    client_max_body_size 1m;
    proxy_connect_timeout 2s;
    proxy_read_timeout 10s;
}
```

`$remote_addr` remplace ici toute chaîne envoyée par le client ; si plusieurs proxies approuvés existent, la politique de chaîne doit être documentée. Les limites réduisent l'épuisement sans promettre une protection DDoS complète. `proxy_pass` avec ou sans URI finale a une sémantique différente : vérifier le chemin réellement transmis.

2. Le cycle TLS complet

Un certificat relie une clé publique à des noms et une période, sous signature d'une autorité. Le déploiement comprend : génération/protection de clé, demande, émission, chaîne intermédiaire, installation, validation depuis le client, surveillance d'expiration, renouvellement et révocation/remplacement après compromission. Un certificat auto-signé de laboratoire est acceptable si le client fait explicitement confiance à cette autorité locale ; `-k` supprime une vérification essentielle et ne constitue pas une preuve.

TLS 1.3 protège confidentialité et intégrité du canal et authentifie le serveur selon la validation. Il ne corrige ni BOLA, ni XSS, ni données sensibles dans les logs. Le TLS externe peut se terminer au proxy ; le flux interne mérite sa propre protection selon la menace. HSTS demande au navigateur de rester en HTTPS après réception de la politique. Ne pas tester `includeSubDomains/preload` sur un domaine partagé ou de production sans gouvernance : une erreur peut couper des sous-domaines.

```
openssl s_client -connect 127.0.0.1:8443 -servername shop.test \
  -verify_hostname shop.test -CAfile lab-ca.pem </dev/null
```

La commande vérifie chaîne et nom avec une CA de laboratoire. Noter protocole, suite, dates et SAN ; ne jamais publier la clé privée.

3. Durcir les conteneurs en couches

Une image minimale réduit paquets et surface, mais « distroless » ne remplace pas les mises à jour. Épingler une image par version/digest, exécuter comme utilisateur non root, rendre le système de fichiers read-only, retirer les capacités, définir `no-new-privileges`, limiter CPU/mémoire/PIDs, isoler les réseaux et ne publier que `127.0.0.1:port`. Les secrets ne vont ni dans l'image ni dans les variables affichées par des logs ; utiliser un mécanisme de secret avec permissions minimales.

```
api:
  read_only: true
  user: "10001:10001"
  cap_drop: ["ALL"]
  security_opt: ["no-new-privileges:true"]
  tmpfs: ["/tmp:size=16m,noexec,nosuid"]
  networks: [frontend, backend]
```

Le service doit toutefois pouvoir écrire là où c'est nécessaire. Une correction aveugle qui casse health check, certificats ou répertoire temporaire n'est pas validée. Les sockets Docker et modes privilégiés donnent un pouvoir proche de l'hôte : ils restent absents du lab.

4. Séquence de vérification

Comprendre les deux connexions, la chaîne de confiance et les écritures nécessaires. **Observer** ports, certificat, headers et identité du processus. **Reproduire sûrement** sur ShopLab : envoyer un faux `X-Forwarded-For`, tenter HTTP, écrire hors tmpfs. **Corriger** au proxy/Compose. **Vérifier** que le faux

header est écrasé, que TLS/nom passent, que l'API reste saine, que l'erreur applicative conserve son statut au lieu de devenir un 200 générique. **Documenter** configuration, sortie expurgée, versions et tests positifs/négatifs.

Idées fausses à corriger

- « Derrière le proxy » ne signifie pas « en-têtes fiables ».
- Un cadenas valide le canal et le nom, pas l'honnêteté de l'application.
- Un conteneur n'est pas une machine virtuelle ni une frontière absolue.
- Masquer tous les 5xx par `200 {error:...}` détruit la sémantique et la détection.

Vérification des connaissances

1. Qui doit écraser `X-Forwarded-For` venant d'Internet ? **Le premier proxy de confiance.**
2. Pourquoi éviter `curl -k` comme validation ? **Il désactive la vérification du certificat et du nom.**
3. Quel binding garde le lab local ? `127.0.0.1:port:port`, pas `0.0.0.0`.
4. Pourquoi retester le cas nominal après `read_only` ? **Le service peut avoir des écritures légitimes ; le durcissement doit rester fonctionnel.**

Références normatives et officielles

- OWASP ASVS `v5.0.0-4.1.3, 3.4.1, 12.1.1, 12.2.1-12.3.5, 15.2.5` : <https://github.com/OWASP/ASVS/tree/v5.0.0>
- RFC 7239, sections 4–7, champ `Forwarded` et sécurité : <https://www.rfc-editor.org/rfc/rfc7239>
- RFC 8446, sections 2 et 4, TLS 1.3 : <https://www.rfc-editor.org/rfc/rfc8446>
- RFC 6797, sections 6–8 et 11, HSTS et considérations : <https://www.rfc-editor.org/rfc/rfc6797>
- Nginx, modules proxy et SSL officiels : https://nginx.org/en/docs/http/nginx_http_proxy_module.html et https://nginx.org/en/docs/http/nginx_http_ssl_module.html
- Docker, bonnes pratiques de sécurité Engine : <https://docs.docker.com/engine/security/>

S12 — PostgreSQL : rôles, chiffrement, sauvegarde et audit

Durée théorie/démonstration : 90 min · Compétence : C10

Prérequis et résultats attendus

Savoir écrire une requête paramétrée (S07) et appliquer le moindre privilège (S06). L'étudiant saura séparer propriétaire, migration, application et lecture ; raisonner sur `GRANT` et privilèges par défaut ; protéger connexions et données ; réaliser un couple sauvegarde/restoration vérifié ; configurer un audit proportionné.

1. Les rôles sont des identités techniques

Dans PostgreSQL, un rôle peut se connecter ou porter des privilèges. L'application ne doit pas être superuser, propriétaire de base ni rôle de migration. Une séparation simple :

- `shop_owner` possède schémas/objets mais ne sert pas au runtime ;
- `shop_migrator` applique une migration contrôlée ;
- `shop_app` se connecte et exécute seulement les opérations nécessaires ;
- `shop_readonly` sert à un reporting limité ;
- l'opérateur de sauvegarde possède les droits documentés, sans devenir le compte applicatif.

```
REVOKE ALL ON DATABASE shop FROM PUBLIC;
REVOKE CREATE ON SCHEMA public FROM PUBLIC;

GRANT CONNECT ON DATABASE shop TO shop_app;
GRANT USAGE ON SCHEMA shop TO shop_app;
GRANT SELECT, INSERT, UPDATE ON shop.orders TO shop_app;
GRANT USAGE, SELECT ON SEQUENCE shop.orders_id_seq TO shop_app;
```

Chaque ligne retire une capacité implicite ou ajoute une capacité précise. Les séquences ont leurs propres droits. `DELETE`, DDL et lecture de tables d'identité restent absents si l'application ne les exige pas. Tester avec `SET ROLE` ou une connexion réelle ; lire le SQL ne prouve pas l'état effectif.

2. Les nouveaux objets et la sécurité au niveau ligne

Un grant actuel ne s'applique pas automatiquement aux futures tables. Le propriétaire configure `ALTER DEFAULT PRIVILEGES` pour le rôle qui créera effectivement les objets. Une erreur fréquente est d'exécuter cette commande sous le mauvais rôle.

Row-Level Security (RLS) peut ajouter une frontière dans la base : une policy filtre les lignes selon un contexte fiable. Elle complète, sans remplacer, l'autorisation applicative ; le propriétaire et certains rôles peuvent contourner RLS. Le contexte de tenant ne doit jamais provenir d'une variable SQL librement modifiable par le client.

3. Chiffrement et secrets

TLS protège le flux API→PostgreSQL si le client vérifie CA et nom ; `sslmode=require` chiffre mais ne garantit pas toujours l'identité comme `verify-full`. Le chiffrement disque/volume protège surtout un support perdu ; un compte DB autorisé voit les données déchiffrées. Pour une donnée très sensible, définir menace, gestion de clés, rotation et recherche avant de choisir un chiffrement applicatif ou colonne.

Le mot de passe DB est un secret court-vivant si possible, injecté par un mécanisme de secrets, jamais commité dans `.env`. Sa rotation doit être testée sans indisponibilité prolongée.

4. Une sauvegarde n'existe qu'après restauration testée

```
pg_dump --format=custom --no-owner --file=shop.dump shop
createdb shop_restore_test
pg_restore --exit-on-error --clean --if-exists \
  --dbname=shop_restore_test shop.dump
```

Le format custom est inspectable/restaurable sélectivement. `--no-owner` facilite une restauration sous rôles reconstruits ; la stratégie exacte dépend du besoin. La restauration se fait dans une base **isolée**, jamais sur la base active du lab sans reset explicite. Vérifier schéma, nombre d'objets, invariants métier et droits, puis détruire proprement la base de test.

Définir RPO (perte de données tolérée) et RTO (durée de reprise), fréquence, rétention, chiffrement, stockage séparé et test périodique. Une copie sur le même volume ne protège pas une perte du volume. La sauvegarde contient des données sensibles : accès et suppression sont audités.

5. Journaliser sans tout enregistrer

PostgreSQL peut journaliser connexions, échecs, DDL, durée et requêtes selon configuration. `log_statement=all` en production peut capturer secrets, volume énorme et données ; préférer une politique ciblée, paramètres expurgés, `application_name` et corrélation. L'extension pgaudit peut compléter l'audit selon la gouvernance, mais installation et volume doivent être évalués.

Boucle : comprendre la matrice de droits/RPO-RTO ; observer `\du`, `\dp`, transport et logs ; reproduire une opération permise et une interdite ; corriger grants/connexion ; sauvegarder puis restaurer isolément ; vérifier données **et privilèges** ; documenter hashes, tailles, heures UTC et cleanup.

Idées fausses

- Le propriétaire d'une base n'est pas un compte runtime normal.
- Une sauvegarde réussie sans restauration testée n'est pas une preuve de reprise.
- TLS en transit et chiffrement au repos répondent à des menaces différentes.
- Journaliser toutes les requêtes peut créer une nouvelle fuite.

Questions de contrôle

1. Pourquoi séparer migrator et app ? **Le runtime n'a pas besoin de DDL ; la compromission a moins d'impact.**
2. Que faut-il tester après restore ? **Intégrité fonctionnelle, objets, droits, accès et procédure de cleanup.**
3. `sslmode=require` suffit-il à authentifier le serveur ? **Préférer `verify-full` avec confiance et nom vérifiés.**
4. Les default privileges sont définis pour qui ? **Pour le rôle créateur concerné, dans le schéma/base visés.**

Références officielles

- PostgreSQL current, rôles et attributs : <https://www.postgresql.org/docs/current/user-manag.html>
- PostgreSQL current, privilèges et `GRANT` : <https://www.postgresql.org/docs/current/ddl-priv.html> et <https://www.postgresql.org/docs/current/sql-grant.html>
- PostgreSQL current, RLS : <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>
- PostgreSQL current, `pg_dump/pg_restore` : <https://www.postgresql.org/docs/current/app-pgdump.html> et <https://www.postgresql.org/docs/current/app-pgrestore.html>
- PostgreSQL current, support SSL et vérification client : <https://www.postgresql.org/docs/current/ssl-tcp.html> et <https://www.postgresql.org/docs/current/libpq-ssl.html>
- PostgreSQL current, journalisation : <https://www.postgresql.org/docs/current/runtime-config-logging.html>
- OWASP ASVS `v5.0.0-1.2.4`, `12.3.1-12.3.4`, `13.2.1-13.2.2`, `14.1.2`, `16.2.5` : <https://github.com/OWASP/ASVS/tree/v5.0.0>

S13 — DevSecOps, chaîne logicielle et décisions de livraison

Durée théorie/démonstration : 90 min · Compétence : C11

Prérequis et résultats attendus

Comprendre les vulnérabilités S06–S12 et utiliser Git. L'étudiant saura placer secrets, SAST, SCA, DAST et tests de sécurité dans un pipeline ; maintenir un SBOM ; trier un finding ; définir une exception versionnée et expirante ; distinguer « outil exécuté » de « gate de sécurité réussi ».

1. La sécurité suit le changement

DevSecOps ne signifie pas acheter un scanner. Il rend les attentes reproductibles à chaque changement : format/lint, tests unitaires de contrôles, secret scan, SAST, SCA/SBOM, build déterministe, analyse d'image, déploiement éphémère local, DAST borné, export et preuve. Les contrôles rapides donnent un feedback précoce ; les plus coûteux passent après un build fiable.

NIST SSDF regroupe la démarche : préparer l'organisation (PO), protéger le logiciel (PS), produire un logiciel bien sécurisé (PW), répondre aux vulnérabilités (RV). Un pipeline concrétise certaines pratiques mais ne remplace ni modèle de menace ni revue humaine.

2. Ce que chaque signal voit — et ne voit pas

Signal	Trouve bien	Limite structurante
secret scan	motifs/entropie dans arbre et historique	faux positifs, secrets inconnus ou hors dépôt
SAST	flux/sinks et motifs dans code	contexte runtime et logique métier incomplets
SCA	composants/CVE/licences	exploitabilité et composants non inventoriés
DAST	comportement d'une instance	couverture dépend des routes/états/authentification
image scan	paquets OS/langage présents	configuration runtime et vulnérabilités métier

Un SCA a besoin d'un lockfile et d'un inventaire transitif. Un SBOM (CycloneDX ou SPDX) décrit des composants ; il n'affirme pas qu'ils sont sûrs. Épingler les actions CI et dépendances, vérifier provenance et protéger les artefacts réduit la compromission de chaîne.

3. Secrets : empêcher, détecter, révoquer

Un secret exposé dans Git doit être **révoqué/roté**, puis retiré des usages ; supprimer le fichier au dernier commit ne retire pas l'historique. Le pipeline utilise permissions minimales, environnements protégés et credentials courts. Les PR non fiables ne reçoivent pas de secrets. Les logs et artefacts sont aussi scannés.

```
permissions:
  contents: read

jobs:
  security:
    steps:
      - uses: actions/checkout@<FULL_COMMIT_SHA>
      - run: ./tools/check-secrets.sh
      - run: ./tools/run-tests.sh
```

Le SHA complet rend l'action tierce immuable à cette révision ; il faut un processus de mise à jour. Le script retourne un code non nul sur finding non accepté : imprimer « vulnerabilities found » puis forcer `exit 0` crée un faux vert.

4. Triage et exception contrôlée

Un finding se confirme avec composant/ligne, version, chemin d'exécution, préconditions, impact et preuve sûre. Les statuts possibles sont confirmé, faux positif démontré, non applicable démontré, corrigé/retesté ou accepté temporairement. Une exception contient identifiant du finding, portée minimale, justification, contrôle compensatoire, propriétaire, approbateur, date d'expiration et ticket. Elle est versionnée ; l'expiration fait échouer le gate.

```
id: CVE-YYYY-NNNN
scope: package==1.2.3
reason: "chemin vulnérable non importé; preuve: TEST-S13-08"
compensating_control: "feature disabled"
owner: "À confirmer"
expires: 2026-10-15
```

Le niveau CVSS aide mais ne décide pas seul : une vulnérabilité critique non atteignable et un secret actif « medium » n'ont pas la même urgence réelle. La politique définit SLA par risque et bloque ce qui dépasse.

5. Boucle de pipeline

Comprendre la menace de chaque étape et son code de sortie. **Observer** une exécution baseline liée au commit. **Reproduire sûrement** avec un marqueur secret factice explicitement allowlisté dans un fixture ou une dépendance vulnérable dédiée au lab. **Corriger** source/version/policy. **Vérifier** que le finding disparaît et qu'un contrôle synthétique connu déclenche encore le gate. **Documenter** commit, outils/versions, commandes, rapports, exceptions et limites.

Un rapport ancien n'est jamais recyclé après changement. Distinguer : [testé ici](#), [validé en CI](#), [documenté](#) – [validation manuelle requise](#).

Idées fausses

- Zéro finding ne prouve pas zéro vulnérabilité.
- Un [continue-on-error](#) transforme un contrôle en information, pas en gate.
- Mettre un secret en variable CI ne le rend pas sûr si le job non fiable peut l'imprimer.
- Mettre à jour aveuglément toutes les dépendances peut casser le produit ; corriger puis tester.

Vérification des connaissances

1. Que faire d'un secret commité puis supprimé ? **Le révoquer/rotater, analyser l'usage et traiter l'historique selon procédure.**
2. Pourquoi générer un SBOM ? **Inventorier versions/transitifs pour risque, réponse et traçabilité.**
3. Quand une exception est-elle acceptable ? **Justifiée par preuve, approuvée, minimale, compensée et expirante.**
4. Comment tester le test ? **Introduire un cas synthétique contrôlé qui doit faire échouer le gate.**

Références officielles

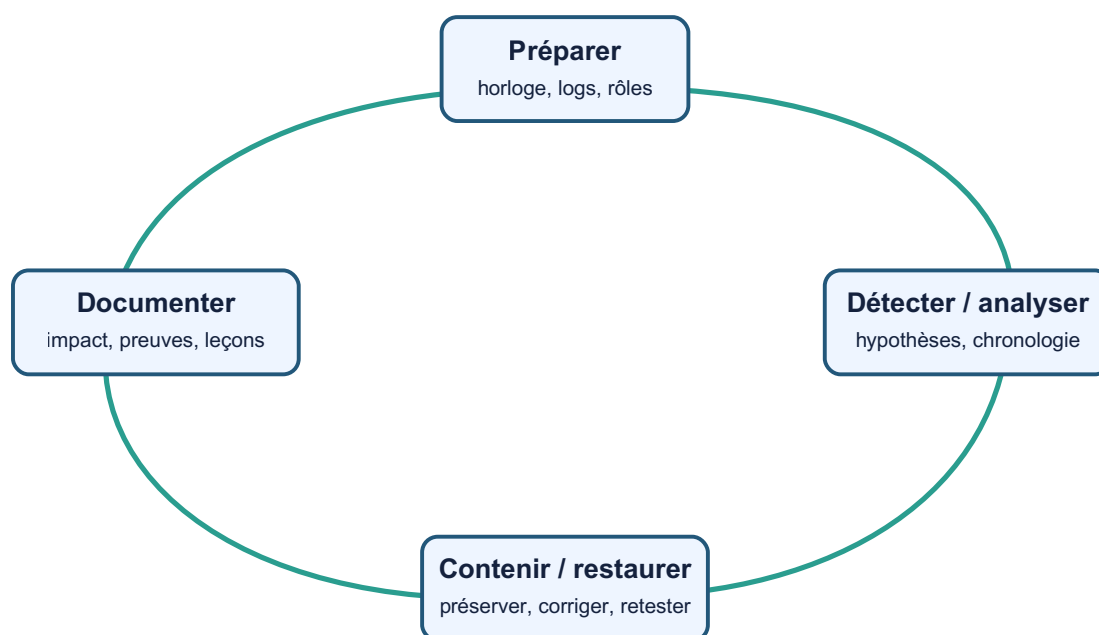
- NIST SP 800-218 v1.1, SSDF, pratiques [PO](#), [PS](#), [PW](#), [RV](#) : <https://csrc.nist.gov/pubs/sp/800/218/final>
- OWASP ASVS [v5.0.0-13.1.4](#), [13.3.1-13.3.4](#), [15.1.1-15.2.5](#) : <https://github.com/OWASP/ASVS/tree/v5.0.0>
- SLSA v1.2, niveaux et provenance de build : <https://slsa.dev/spec/v1.2/>
- CycloneDX Specification, SBOM : <https://cyclonedx.org/specification/overview/>
- SPDX 3.0, spécification officielle : <https://spdx.github.io/spdx-spec/v3.0/>
- GitHub Actions, sécurité et durcissement : <https://docs.github.com/en/actions/security-for-github-actions/security-guides/security-hardening-for-github-actions>

S14 — Observabilité, incident, remédiation et rapport

Durée théorie/démonstration : 90 min · Compétences : C01, C12

Prérequis et résultats attendus

Mobiliser tout le parcours S01–S13. L'étudiant saura distinguer logs, métriques et traces ; définir des événements de sécurité utiles ; construire une chronologie ; trier et contenir un incident sans détruire la preuve ; corriger, retester et écrire un rapport professionnel avec niveau de confiance et risque résiduel.



1. Trois signaux, une même corrélation

Un **log** décrit un événement discret ; une **métrique** agrège une mesure dans le temps ; une **trace** relie les spans d'une opération distribuée. Aucun signal ne suffit seul. Un `request_id` suit la requête ; un `trace_id` relie les services ; un identifiant utilisateur pseudonymisé relie des décisions sans inscrire email ni token.

```
{  
  "ts": "2026-09-18T09:14:22.481Z",
```

```
"event": "authorization.denied",
"request_id": "lab-s14-0042",
"subject_hash": "u:8f3a...",
"action": "order.read",
"object_type": "order",
"route": "/api/orders/{id}",
"result": "deny",
"reason": "not_owner"
}
```

Le temps est UTC explicite ; la route est normalisée ; aucun `order_id`, cookie ou corps complet n'est nécessaire. Encoder les champs évite l'injection de log. Contrôler accès, intégrité, rétention et suppression. Synchroniser les horloges : sans cela, la chronologie ment.

Métriques utiles : taux de 401/403/429, 5xx, latence, saturation, échecs login par compte et contexte. Une alerte combine seuil, fenêtre, baseline et runbook. Un pic de 403 peut être une attaque, un bug frontend ou une policy déployée : l'alerte lance une enquête, elle ne prononce pas le verdict.

2. Trier avant d'agir

Déclencheur → confirmer le signal → définir portée/impact → attribuer sévérité et propriétaire → contenir proportionnellement. Préserver les données volatiles utiles avant de redémarrer si c'est sûr. Ne pas exécuter un script inconnu trouvé sur le système. Toute action est horodatée et attribuée.

Questions : quelles identités, objets, périodes et versions ? L'événement continue-t-il ? Le secret doit-il être révoqué ? L'isolation risque-t-elle d'aggraver l'impact ? Qui a autorité pour couper le service ? Dans le lab, ces rôles sont simulés ; en organisation, le plan et les contacts sont établis avant l'incident.

3. Contenir, éradiquer, restaurer et apprendre

Contenir limite l'impact : révoquer un token, désactiver une route, isoler un conteneur. Éradiquer retire la cause : corriger l'autorisation, mettre à jour un composant, supprimer une persistance. Restaurer remet en service depuis un état digne de confiance. Puis surveiller la récurrence et organiser le retour d'expérience sans blâme simpliste.

La correction suit la même preuve que le cours : test avant, modification minimale, même test négatif, test positif, observation des logs et risque résiduel. Une règle WAF temporaire peut contenir ; elle n'est pas automatiquement l'éradication.

4. Rapport de sécurité utile à deux publics

Résumé exécutif : actif, impact, urgence, décision attendue, sans jargon. Partie technique : scope/ autorisation, environnement/version, préconditions, étapes minimales, observations expurgées, cause racine, impact, remédiation, retest, limites. Ajouter une chronologie UTC et un index de preuves avec SHA-256.

```
Finding WEB-06 – lecture d'une commande d'un autre compte
Portée : ShopLab local, commit abc1234, comptes synthétiques A/B
Observation : A-A = 200 ; A-B = 200 avant correction
Cause : chargement par id sans filtre owner/policy
Correction : policy objet centralisée
Retest : A-A = 200 ; A-B = 403 ; anonyme = 401
Risque résiduel : routes export/invoice à vérifier séparément
Confiance : élevée, reproduit 3/3 ; limites explicitement listées
```

Éviter « application sécurisée ». Dire ce qui a été testé, sur quelle version, avec quel résultat. Séparer faits, hypothèses et recommandations. Un score ne remplace pas l'explication métier.

5. Capstone : la boucle complète

Comprendre scope et architecture. **Observer** baseline et télémétrie. **Reproduire de manière sûre** une anomalie fournie, sans cible externe. **Corriger** la cause. **Vérifier** sécurité, fonctionnalité, détection, reset et cleanup. **Documenter** chronologie, preuves expurgées, décision et limites. Arrêter immédiatement sur secret réel, donnée personnelle, sortie de scope ou instabilité.

Idées fausses

- Plus de logs n'implique pas meilleure détection ; qualité, contexte et protection comptent.
- Un redémarrage peut effacer la preuve et masquer la cause.
- Contenir n'est pas corriger ; corriger n'est pas restaurer ; restaurer n'est pas apprendre.
- Un rapport sans version, scope et retest n'est pas reproductible.

Vérification des connaissances

1. Pourquoi normaliser `/orders/42` en `/orders/{id}` dans les métriques ? **Réduire cardinalité et fuite d'identifiant, agréger le même comportement.**
2. Première action après une alerte ? **Confirmer/qualifier sans action destructrice automatique, selon runbook.**
3. Que contient un risque résiduel ? **Ce qui reste exposé/non testé, ses conditions et son traitement.**
4. Quelle différence containment/remediation ? **Limiter l'incident maintenant versus retirer sa cause.**

Références officielles

- NIST SP 800-61 Rev. 3 (avril 2025), *Incident Response Recommendations and Considerations for Cybersecurity Risk Management* : <https://csrc.nist.gov/pubs/sp/800/61/r3/final>
- NIST SP 800-92, sections 2–4, gestion des logs : <https://csrc.nist.gov/pubs/sp/800/92/final>
- OWASP ASVS v5.0.0–16.1.1, 16.2.2, 16.2.4–16.2.5, 16.3.2–16.4.1 : <https://github.com/OWASP/ASVS/tree/v5.0.0>

- OWASP Logging Cheat Sheet, événements, données à exclure et vérification : https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html
- OpenTelemetry, concepts traces/métriques/logs : <https://opentelemetry.io/docs/concepts/>
- OWASP WSTG 4.2 [WSTG-ERRH-01](#) et [WSTG-ERRH-02](#), erreurs et stack traces : https://wstg.owasp.org/v4.2/4-Web_Application_Security_Testing/08-Testing_for_Error_Handling/